Auxiliary engineering document

# Implementing MeTTaDinC in Cetta

## A Coding-Agent Specification and Work Plan

Companion to the MeTTaDinC specification draft of 4 November 2026
4 November 2026

Developed from the d-calculus and Hyperseed project

**First release.** Strict certificate boundaries first: results bound to the exact objects they are about, one assurance policy for every status, obligations generated from the admitted source. Then a held-syntax frontend driven by the registry; a term-level typed IR; the graded deterministic core with its crisp special case, on an enumerated table backend and a symbolic backend with an explicit admission contract; measured and checked defects with two-sided certificates, splitting trees and proof-logged obligations, all replayed by an independent checker; certificate transport across interface changes. Acceptance: the negative acceptance suite, the 45 first-release claims of the manifest, and Unsupported for the other 78.

**Implementation discipline.** Generate interfaces from the registry instead of reading them off the PDF. Preserve the specified claim kind and scope through parsing, interpretation, compilation, serialization and optimization. Use the efficient classical algorithms within their proved scope. Report unsupported capability as Unsupported.

# Purpose and use

This document hands the implementation of MeTTaDinC to coding agents. It accompanies the language specification of the same date [1], which is normative for every semantic claim, and the validation package, which is normative for every interface: the registry file `registry/mettadinc_registry.json`, the fixture modules in `fixtures/`, the claims manifest file `manifest.json`, the oracle `oracle/verify_mettadinc.py` with its results, the static checker `check_fixtures.py`, and the pinned versions in `DEPENDENCIES.json`. Here are the invariants an implementation must keep, the intermediate representation and backends, the default algorithms and their scope, the certificate formats and their replay, the trusted boundary of the symbolic path, the extension points of Cetta at a pinned revision, and a sequence of milestones whose acceptance is stated in manifest claims.

The order of work is: build a small implementation whose certificate boundaries are strict, so that every result is bound to the objects it is about and every status means one thing; generate the full frontend from the registry and load the fixtures; implement the graded deterministic core on tables; add the symbolic backend with its obligation generator and proof pipeline so that the forty-bit fixtures pass without enumeration and the adversarial candidate is rejected; establish independent replay and differential validation; then add stochastic dynamics and linear realizations, nondeterminism and simulation, the composition kernel with both kinds of hive, and the Hyperseed, discovery and integration libraries. A release whose scope is small but exact is useful. The examples already exercise more forms than a first implementation can make reliable, which is why the work starts with the certificate boundaries.

### Coding-agent entry checklist

1. Read the specification, this guide and the package README of the same date. Run `python3 oracle/verify_mettadinc.py` and `python3 check_fixtures.py`; both must succeed before any work starts, and both must still succeed when it ends.
2. Record the Cetta revision and read the repository's build and contribution instructions before editing. Record the registry digest you build against.
3. Identify your milestone, its manifest claims, owned files, shared IR interfaces and required negative cases. A change to the registry, a certificate format or canonical serialization is a coordinated change: it updates the registry, the fixtures, the manifest and the checker together, and dependent work waits for it.
4. Implement the smallest complete path from source through admission, execution and replay. Never hard-code fixture answers, and never resolve an interface question from a PDF listing when the registry or a fixture answers it.
5. Keep the negative acceptance suite of Section 3.1 passing at every step: each of its fixtures is a small change that must make a successful result fail with a precise status.
6. Return reproducible commands, structured results, replay evidence, resource records, the manifest claims covered and the remaining unsupported surface. Performance claims need measurements with unchanged semantics and arithmetic.

# Contents

**1 Implementation contract** **3**

1.1 Authority and scope . . . . . 3

1.2 Semantic invariants . . . 3
1.3 Registry-driven handoff . . . 4
1.4 A term-level typed IR . . . 5
1.5 Backends . . . 6
1.6 Results, projections and status rules . . . 7
1.7 Certificate formats and independent replay . . . 7
1.8 Symbolic admission, the obligation generator and the proof pipeline . . . 11
1.9 Default algorithms and their scope . . . 11
1.10 Resources and incrementality . . . 13
1.11 Estimates and their conversions . . . 13

**2 Cetta integration** **13**
2.1 Pin and inspect the host . . . 13
2.2 Source map . . . 14
2.3 Admission . . . 16
2.4 Ownership and persistence . . . 16
2.5 Dependencies and entry criteria . . . 17
2.6 Reference path, specialized paths and replay . . . 17
2.7 Supercompilation, generated packs and the LLVM emitter . . . 17
2.8 Host acceptance . . . 18

**3 Milestones, fixtures and acceptance evidence** **18**
3.1 M0: strict certificate boundaries . . . 18
3.2 M1: registry, syntax, staging and result plumbing . . . 20
3.3 M2: the graded deterministic core on tables . . . 21
3.4 M3: symbolic carriers and incrementality . . . 21
3.5 M4: stochastic dynamics, linear realizations and estimates . . . 22
3.6 M5: nondeterminism and simulation . . . 23
3.7 M6: composition kernel, hives, merges, graded kernels and contracts . . . 23
3.8 M7: Hyperseed, observer discovery and integration . . . 23
3.9 Golden results and adversarial mutations . . . 24
3.10 Handoff and definition of done . . . 27

# 1 Implementation contract

## 1.1 Authority and scope

The specification is authoritative for semantics and the registry for interfaces. When an example, an optimization or a host convenience conflicts with either, keep the semantics and the registry and report the conflict. Never weaken a claim to make a fixture pass. No MeTTaDinC implementation exists yet. The target is a reflective MeTTa sublanguage hosted first in Cetta, with a portable semantic core that does not depend on a particular evaluator's rewrite order. Unrestricted MeTTa evaluation, neural training, global probabilistic inference, a general theorem prover and the whole of Hyperseed are outside the first release.

## 1.2 Semantic invariants

Types and result schemas must keep the following distinctions visible.

1. Claim kinds are distinct types: current adequacy, update compatibility, behavioral (bisimulation) adequacy, trace adequacy, simulation, exact induction, approximate induction and linear realization. Evidence for one never satisfies a consumer of another without a registered theorem; in particular a positive behavioral defect does not refute trace equality.
2. Observation equality is `Literal` or `Observer`. A zero current defect under a non-separated value tolerance yields decoders into the zero-distance quotient only, and the result says so.
3. A measurement (`Measure*`, `Defects`) is Checked when its interval is computed to the requested precision, whatever the value. A check (`Check*`) compares the interval with `Within`, whose default is 0. `Output` projects only from Checked results.
4. `InduceExact` requires current adequacy and update compatibility with defect zero, and literal congruence of the code fibers, checked on code values whatever the code tolerance (claims C01–C03); it returns a quotient arrow. `InduceApprox` returns a representative machine, its section (default `Center`), radius, arrow defect and a priori bound.
5. The horizon `(Steps h)` covers the initial observation and $h$ transitions; its tolerance is $\Phi^{h+1}(1)$.
6. A transition law, a tolerance, a checked transfer and a host result bag are different types. A bag becomes a set, multiset or law only under the interpretation the interface declares; a transfer becomes a kernel only through the registered `KernelOf` construction.
7. Arrow kinds have distinct types and registered composition laws: functions for quotient arrows and homomorphisms, additive defects for graded arrows and transfers, $(\lambda_L\lambda_K, \rho_L\lambda_K + \lambda_L\rho_K)$ for linear arrows, $(L_2L_1, L_2\eta_1 + \eta_2)$ for affine contracts. Composing across kinds without a registered conversion is InvalidInput.
8. A signed linear realization is not a machine. A replayed positive candidate bounds the positive dimension from above only.
9. Kernel hives and transfer hives are different types with different coherence claims.
10. A symbolic carrier is admitted only with its obligations discharged; type-checking its terms is not admission.
11. A prior-dependent averaged model is an aggregate, not a quotient.
12. Probabilities, tolerances and defects, p-bit pairs, report availability, error bounds, operator norms and costs never convert implicitly; an estimate becomes PLN evidence only by a conversion that preserves its estimand and dependence assumptions (Section 1.11).

13. Exact claims are about admitted models; population claims need their own evidence and status.
14. Shared and fresh reads of random or nondeterministic variables are declared, never inferred.
15. Unmeasured correspondences in a hive are marked, never filled with zero.
16. Every result carries its binding: module digest and subject names, source and target model identities, the checked interface with its value tolerances, and its scope (initial states, reachable-state scope, horizon, discount, policy class). Every operation that consumes a result validates the binding first, and a mismatch is InvalidInput naming the field, in every release.
17. One assurance policy assigns every status: Checked only on the grounds `Exact` and `Replayed`, Refuted only on a replayed `Witness`, Conditional on `(TrustedBackend solver version logic obligation-digest)` for an unreplayed backend answer, with `agreed_by` recorded as evidence, on `(Assumption name digest)` for an assumption the source declares, and on `(EstimatedPremise result-digest delta)` for a premise that is an estimate; derived results inherit the union of their premises' assumptions, and consumers accept only what their `Accepts` declaration lists.
18. A distance value enters a result only through the registered certificate kinds, whatever algorithm computed it, and an exact measurement needs a lower and an upper certificate that meet; when the distance is a supremum that no registered certificate attains, the answer is Unknown with the interval. Trace-event and policy-tree tests are undiscounted probabilities and bound a behavioral distance at discount one only. On a stochastic machine a trace refutation over observation-based policies needs a policy tree; a word witness never certifies it.

Five regression cases protect these invariants from the start, each with its manifest claims. First, states $0, 1, 2$ with constant observations, encoding $\{0, 1\} \mid \{2\}$ and update $0 \mapsto 0$, $1 \mapsto 2$, $2 \mapsto 2$: the behavioral defect is zero while the update defect is one (H09, H10). Second, the posterior summary of the receipt memory: it is not compatible with the union merge, and a merge of posteriors as independent evidence double counts shared receipts (P10–P13). Third, the forty-bit zero machine: a sampled relevance screen drops the tail bits, and the resulting four-class encoding has behavioral defect one (F07). Fourth, the adversarial zero-tail candidate, which passes every local check and fails coverage (F05). Fifth, for the first stochastic milestone, the seven-state chain, where the behavioral check is refuted by a modal test while the trace check passes (S03, S05).

### 1.3 Registry-driven handoff

The registry fixes, for every declaration head, its positional arguments, fields, field kinds and which fields are required or repeatable; for every request, its library, positional arguments, keyed arguments with kinds and defaults, result type, statuses beyond the base statuses, supported dynamics and carrier backends, and the dynamics it supports in the first release; for every result type, its fields; and the first-release capability manifest. Build the frontend from it:

- generate, or drive from tables, the declaration parser, the argument checker and the result schemas, so that a registry change propagates mechanically;
- give every request the base statuses Conditional, Unknown, Unsupported and InvalidInput in addition to its registered ones;
- apply registered defaults at admission and record them in the claim, so that `(CheckBehavioral R)` and `(CheckBehavioral R (Within 0) (Discount 1) (Horizon`

`AllFiniteWords))` have the same identity;

- answer Unsupported, never InvalidInput, for a well-formed request outside the running release's manifest whose premises pass the binding rules, and InvalidInput for a malformed request or one whose premises fail them;
- treat a method declared in a module as a request whose result type is its declared `Returns` type, which must equal the result type of its body's final request.

The checker `check_fixtures.py` is the reference reading of the registry for static questions: an implementation's admission must accept every fixture module it accepts and reject the mutations it rejects (Section 3.9).

The first-release manifest covers deterministic dynamics, crisp and Łukasiewicz values, enumerated and symbolic carriers with the `BitVectors` primitives, the core requests and the `GradedKernels` library: `BehavioralTolerance`, `Bracket`, `CheckAbstractionMap`, `CheckBehavioral`, `CheckBlockPartition`, `CheckNow`, `CheckTrace`, `CheckTransitive`, `CheckUpdates`, `Defects`, `Delta`, `InduceApprox`, `InduceExact`, `MeasureBehavioral`, `MeasureNow`, `MeasureTrace`, `MeasureUpdates`, `MinimizeForQueries`, `PathClosure`, `Refine`, `SamePartition`, `Then`, `Threshold`, `WeakestWithin` (24 requests, generated from the registry).

## 1.4 A term-level typed IR

Define a portable typed IR before any specialized backend. Maps are terms over registered primitives; a table is one representation of a term, chosen when the carrier is enumerated. An IR built around dense state identifiers cannot later carry symbolic maps, so dense identifiers belong to the table backend only.

| IR object | Required information |
|---|---|
| Module | Registry digest; syntax and semantics versions; profile coordinates; imported library versions; canonical source identity; declarations; dependency graph. |
| Carrier | Element type; one of: enumerated (ordered elements, source-to-index maps), symbolic (bit width or product structure, admitted predicate, discharged admission obligations), reachability scope (machine, initial set, inductive invariant, reachability witnesses when needed), snapshot query (snapshot identity, pattern, key variable). |
| Map | Domain and codomain; admitted term; effect class; totality, exclusivity and closure evidence; optional materializations (table, decision diagram, matrix over a field) with the term they came from. |
| Interface | Query labels with value types, value tolerances and separation flags; action labels with update kinds; availability guards; merge labels; read annotations. Its identity covers all of these. |
| Machine | Carrier, interface, observation maps, update maps or kernels. |
| Linear realization | Dimension, operators, projectors, final vector, normalization evidence. |
| Tolerance, relation | Carrier pair; algebra; entries (dense, sparse or symbolic); properties established (reflexive, symmetric, metric, crisp, separated) rather than assumed. |

| IR object | Required information |
|---|---|
| Kernel | Shared with the MeTTaProN kernel IR once pinned: carriers, exact or interval masses, normalization status, read annotations, provenance. |
| Representation | Source interface identity; encoding map; code tolerance; image carrier. |
| Arrow | Kind, source and target identities, data (map, relation, matrix, or pullback and pushforward), grade (0, defect, $(\lambda, \rho)$ or $(L, \eta)$), premise certificates. |
| Hive | Kernel hive: members, correspondences with measured or unmeasured status, weights. Transfer hive: members, edges, cycles, gauges. |
| Observer program | Held typed AST, bindings, generated parameters, evaluator identity, optional invariance derivation. |
| Result, certificate | Status; claim (kind, bound or precision, equality); binding (module digest, subject names, source and target model identities, interface identity with value tolerances); scope (initial states, reachable-state scope, horizon, discount, policy class); payload; estimate (for Estimated results and results derived from them); interval and certificates; evidence with certificate and stream digests and `agreed_by`; assumptions; assurance; method; dependencies; resource record. |

Witnesses are returned in terms of original carrier elements, never internal identifiers. Partitions are compared as equivalence relations, never by block labels; canonical block order is by least source element. Any change to an observation, value tolerance, update set, arithmetic algebra, cost model, read annotation, gauge or evidence convention creates a new interface identity. Canonical serialization respects lexical binding and alpha-equivalence and never depends on pointers, arena order or a live mutable space.

## 1.5 Backends

| Backend | Computes | Certificates produced |
|---|---|---|
| Table | Everything on enumerated carriers: group-by checks, refinement, distances, closures, induced machines. | Witnesses, stability tables, splitting trees, couplings, matchings. |
| Decision diagrams | Symbolic signature refinement, image and preimage computation, reachability, block predicates. | Block predicates and invariants; obligations handed to the SAT path for proof. |
| SAT with proof logs | Admission obligations, two-copy update-compatibility queries, constancy and stability. | Satisfying assignments as witnesses; DRAT converted to LRAT and checked by `cake_lpr` (Section 1.8). |
| Linear algebra over $\mathbb{Q}$ and $\mathbb{F}_p$ | Krylov iteration for trace equivalence, linear realizations and observability; intertwining residuals and norms; Hom dimensions; conjugators. | Bases, nonsingular minors, matrices, residual bounds, rank computations. |

| Backend | Computes | Certificates produced |
|---|---|---|
| Linear programming | Kantorovich lifts, stochastic behavioral distances, least coherent hive refinements. | Couplings (upper bounds), dual test functions or modal formulas (lower bounds). |
| Snapshot group-by | Functional-dependency checks over MORK or PathMap snapshots. | Group tables and violating pairs. |

`(Backend Auto)` chooses by carrier kind and map structure: linear maps over a finite field go to the linear backend, enumerated carriers below a size threshold to tables, symbolic carriers to decision diagrams with SAT for proofs. The choice is recorded in the result's method field and never changes the claim.

### 1.6 Results, projections and status rules

Every result uses the envelope of the specification, with `Value` fields fixed by the request's registered result type. A user-authored atom headed `Checked` is data. `(Output R Value)` returns the whole payload and `(Output R F)` a registered top-level field, where `R` is a derivation name or a result variable bound by `LetResult`; projection from a result that is not Checked yields a dependency diagnostic. Claims, bindings, scopes and defect intervals serialize as

```
(Claim (Kind Behavioral) (Within 0) (Equality Literal))
(Binding (Module "sha256:...") (Subject PosteriorOnly) (Machine Memory)
         (Interface (Digest "...")))
(Scope (Init (All)) (States (All)) (Horizon AllFiniteWords) (Discount 1))
(Defect (Lower (Rat 3 20) (Certificate (WordWitness ...)))
        (Upper (Rat 3 20) (Certificate (Prefixpoint ...))))
(Assumptions) (Assurance Replayed)
```

A result whose verdict came from an SMT solver without a replayed proof carries instead `(Assumptions (TrustedBackend cvc5 1.4.2 QF_LRA "sha256:..."))`, its evidence lists any agreeing backend under `agreed_by`, and its status is Conditional. A measurement with `(Precision p)` is Checked when $u - \ell \leq p$ and Unknown otherwise. A check with bound $e$ is Checked when $u \leq e$, Refuted when $\ell > e$ with the lower-bound certificate as `Witness`, and Unknown otherwise. A malformed probability row is InvalidInput, never a refutation; an interrupted implemented search is Unknown; an operation outside the manifest is Unsupported; a failed prerequisite produces a dependency diagnostic naming it.

### 1.7 Certificate formats and independent replay

Replay consumes frozen semantic data and certificate records, not a compiler session, and never calls the routine that produced the candidate. Table 3 lists the formats.

| Claim | Certificate | Replay |
|---|---|---|
| Current or update failure | Pair, query or action, the two values, codes or class masses | Evaluate both sides. |
| Behavioral failure, deterministic | `WordWitness`: pair, word, query | Run the word from both states and compare. |

| Claim | Certificate | Replay |
|---|---|---|
| Behavioral failure, stochastic or nondeterministic | `ModalTestWitness`: a formula graph with shared subformulas and its two values | Evaluate each node once, exactly, at both states. |
| Trace failure, fixed words | `TraceEventWitness`: pair, action word, observation event, two probabilities | Exact matrix–vector products. |
| Trace failure, observation-based policies (stochastic) | `PolicyTreeWitness`: pair, finite policy tree keyed by observed histories, event on complete histories, two probabilities | Check that every node is keyed by an observed history, that both states run through the same tree, and that a node exists for every history of positive probability; propagate both laws through the tree exactly; compare $\lvert \mathit{Left} - \mathit{Right} \rvert$ with the bound. A fixed word replays as a tree whose actions ignore the observations. |
| Literal-congruence failure (`InduceExact`) | `CongruenceWitness`: pair with equal code value, action, two successor code values (a code value and two masses for laws; two code sets for nondeterminism) | Apply the encoding to both successors and compare the values literally; the code tolerance is not consulted. |
| Graded upper bound, deterministic or stochastic | `Prefixpoint`: kernel $D$, one coupling per pair and action | Check coupling marginals and $D \geq \max(d_0, \gamma\, \mathbb{E}_\theta D)$. |
| Graded upper bound, nondeterministic | `Prefixpoint`: kernel $D$, two matchings per pair and action | Check matching domains and $D \geq \max(d_0, \gamma D)$ on matched pairs. |
| Graded upper bound at horizon $h$ | `LayeredPrefixpoint`: kernels $D_0, \ldots, D_h$, couplings or matchings per layer, pair and action | Check $D_0 \geq d_0$ and $D_{t+1} \geq \max(d_0, \gamma\, \mathbb{E}_{\theta_t} D_t)$ for every layer. |
| Graded lower bound, stochastic, discount below one | `DualPostfixpoint`: kernel $D$; per pair above $d_0$ an action and rational potentials $f, g$ on the two supports | Check $f(u) - g(v) \leq D(u, v)$ on the supports and $D \leq \gamma(\mathbb{E}f - \mathbb{E}g)$; reject the certificate at discount one, where it bounds only the greatest fixed point. |

| Claim | Certificate | Replay |
|---|---|---|
| Exact value (`Precision 0`) | A lower and an upper certificate | Replay both and require equal values; a distance computed by any algorithm enters only this way. Trace-event and policy-tree tests count as lower certificates at discount one only (at discount 1/2 the one-step event test of U04 would exceed the distance). Over all finite words at discount one the only registered lower certificates are tests, so when no test reaches the upper bound the result is Unknown with the interval (claim U01). |
| Stable partition, enumerated | Block map, induced observations and updates | Check factorization and commutation for every state and action; for laws, exact block sums. |
| Symbolic partition | Block predicates, representatives, successor map, invariant for a reachability scope, reachability words | One proof per obligation of Section 1.8, or a citation of the construction rule that discharges it. |
| Coarsest partition | Splitting tree (deterministic) or characteristic formula graph | Evaluate each test on the representatives below it; combine with stability. |
| Linear arrow | $L$, target realization, residual and norm bounds | Recompute residuals and the $\infty$-norm; check $L\eta' = \eta$ and declared positivity. |
| Signed minimum | Test basis with a nonsingular minor, and a realization | Recompute the minor; replay the realization. |
| Positive dimension | Positive candidate; lower bound from the signed minimum or a registered certificate | Replay the candidate; report the interval. |
| Simulation | Relation, initial coverage, one matching move per related pair and move | One membership check per move; coverage of the initial states. |
| Transfer | $(\varphi, \iota)$ and response family | Compare response laws for each target state and source method. |
| Transfer hive | Edge certificates, gauge symmetries, intertwining of the gauges by each transfer, holonomy per declared cycle | Check each symmetry and each conjugation $\iota^{-1} G_j^M \iota = G_i^M$; compose around each cycle; check membership in the gauge. |
| Kernel hive | Correspondences and the closure | Recompute the closure and compare. |

| Claim | Certificate | Replay |
|---|---|---|
| Least coherent refinement, optimality | Refined kernel and a dual solution of the linear program | Check primal and dual feasibility exactly and the equality of the two objective values (for the parity triangle: kept mass 3, dual weights 1/2 on six constraints). |
| Affine contract | Ideal and actual tables, metrics, grade | Check the inequality on every audited pair. |
| Observer invariance | Typed structural derivation | Recheck each rule application. |
| Module isomorphism | Conjugator, or ranks of two Hom systems | Multiply and invert; or recompute the ranks. |
| Estimated claim | Estimator, data digest, protocol, population, selection rule, policy version, estimand | Check the stream digest before reading any row (a mismatch is InvalidInput, with no estimate); recompute the statistic; the probability guarantee stays an assumption about sampling. |
| Derived result (`Then`, lift, safety transfer, rebuild, gate) | Premise digests with their bindings | Validate every premise's binding against the objects the operation names (InvalidInput on a mismatch); recompute the union of the premises' assumptions; a Conditional premise makes the result Conditional, and an Estimated premise makes it Conditional on `(EstimatedPremise result-digest delta)`. |
| Claim about a system from an estimated kernel | The kernel's provenance record and the declared assumption | Check the provenance digest; the exact result about the kernel stays Checked, and read as a claim about the system it is Conditional on `(Assumption name digest)`, accepted only by a consumer that names the assumption. |
| Backend verdict without a replayed proof | `(TrustedBackend solver version logic obligation-digest)`, and `agreed_by` entries | Nothing to replay: check that the obligation digest is the generator's output for the admitted source, record the assumption and the agreeing backends, and assign Conditional. |

Table 3: Certificate formats and their replay.

Replay assigns statuses by the assurance policy of the specification: a validated certificate gives Checked with assurance `Replayed`, an exact computation of the core gives `Exact`, a replayed counterexample gives Refuted with `Witness`, and a backend answer without replay

gives Conditional. Agreement of a second backend is evidence and never upgrades a Conditional result; only a replayed proof discharges a `TrustedBackend` assumption, after which the dependent derivations are evaluated again. A witness alone does not prove shortestness; when a shortest witness is claimed, the certificate also carries the minimal separating sequence from the splitting tree. Stability alone does not prove coarseness; the splitting tree or formula graph does. A proof log certifies only the formula it was produced for, so the replay checker regenerates the formula from the frozen model with the pinned obligation generator and compares it byte for byte before checking the log.

## 1.8 Symbolic admission, the obligation generator and the proof pipeline

*Obligations.* For a carrier predicate $C$, block predicates $B_i$, representatives $r_i$ and a successor map $\sigma$, a symbolic partition certificate needs coverage ($C \Rightarrow \bigvee_i B_i$), disjointness ($C \Rightarrow \neg(B_i \wedge B_j)$), membership ($C(r_i) \wedge B_i(r_i)$), constancy of every query on every block, stability ($C \wedge B_i \Rightarrow B_{\sigma(i,a)} \circ T_a$), and, for symbolic maps, totality and exclusivity of cases and closure ($C \Rightarrow C \circ T_a$). For a reachability scope, universal obligations are proved on an inductive invariant (contains the initial states, closed under the actions), and existential ones—membership, nonemptiness—need reachability words. An IR node that guarantees an obligation by construction, such as the fibers of a total key function, cites the rule in the certificate instead of a proof. The adversarial candidate of fixture `forty_bits.metta` must be refuted on coverage with witness word 4 (claim F05); the same blocks on `Reach(0)` must be accepted (F06).

*The obligation generator is trusted.* It translates admitted terms and the frozen semantics of the registered primitives into CNF. Keep it small and separate from the solvers: one documented encoding per primitive (`BitAt`, `SwapBits`, `RotateRange`, `SetBit`, `Xor`, `Parity`, `Slice`, `IsZero`, the Boolean connectives), Tseitin definitions with a deterministic variable order, and a deterministic clause order so that the same obligation always yields the same bytes. Version it with the registry and record its version and the CNF digest in every certificate. Test it by translation validation: for widths 3 to 12 of every symbolic fixture, decide each obligation both by enumeration and by the generated CNF, and require agreement, including on the mutations. A verified generator is future work; until then this test set is part of the trusted computing base's evidence.

*The proof pipeline is pinned* (`DEPENDENCIES.json`):

```
cadical --no-binary obligation.cnf obligation.drat           # rel-3.0.1
drat-trim obligation.cnf obligation.drat -L obligation.lrat  # 2e3b2dc
cake_lpr obligation.cnf obligation.lrat                      # 033aad5
# expected output: s VERIFIED UNSAT
```

`cake_lpr` reads LRAT and its LPR extension, not DRAT, so the conversion step is required. Only the generator and `cake_lpr` are trusted; the solver and the converter produce candidates and may be replaced, for example by a solver that writes LRAT directly, after the proof fixtures pass again. The `cake_lpr` pin is announced as its final release; adopting the successor checker is a pin change with its own fixture run. For maps linear over $\mathbb{F}_2$, obligations are matrix identities replayed by multiplication.

## 1.9 Default algorithms and their scope

With $n$ states, $m$ transitions and exact arithmetic costs counted separately:

- *Crisp current and update checks*, those of a representation whose code tolerance is the discrete equality, are functional-dependency tests—does the code determine each query, does it determine each successor's code—implemented as hash group-bys with exact

comparison, linear in $n$ plus $m$. Under a two-valued metric code tolerance the same group-bys run on the classes of the tolerance, and `InduceExact` adds the literal-congruence group-by on the code values.

- *Behavioral partition*: Hopcroft or Paige–Tarjan refinement in $O(m \log n)$ [4, 5]; for deterministic machines, a splitting tree with minimal separating sequences [6]; for other functors, characteristic formulas in shared form [15].
- *Crisp behavioral check*: compute $P_{\mathrm{beh}}$, then test the dependency "code determines behavioral class". A failing group yields a witness pair, separated by the test at their lowest common ancestor. The pair-graph search of the specification's definitions is the reference engine only; it needs $\Theta(n^2)$ memory.
- *Stochastic lumping* in $O(m \log n)$ [7, 8], with exact sums over explicit nonzero entries.
- *Graded distances*: for deterministic machines, shortest paths in the pair graph restricted to the pairs a request needs; for labelled Markov chains with one transition function, exact polynomial-time computation, with policy iteration or on-the-fly algorithms in practice [9, 10, 11]; for several actions, exact value iteration for lower bounds and couplings for upper bounds, reported as an interval with no complexity claim.
- *Trace equivalence and linear realizations*: Krylov iteration over the alphabet of action–observation letters with fraction-free elimination [12, 13]; quantitative trace discrepancy by exact enumeration through a declared horizon, or as an interval bounded above by a behavioral certificate at discount one.
- *Nondeterminism*: bisimulation by relational refinement, simulation preorders in $O(mn)$ [14], distinguishing formulas as in [15].
- *Symbolic refinement*: signature refinement over decision diagrams as in Sigref and Storm [16, 17], preceded by a cone-of-influence pass and, when declared, reachability.
- *Closure*: max–Łukasiewicz Floyd–Warshall in $O(n^3)$.
- *Frontier search*: exhaustive over partitions for small carriers; heuristic otherwise, with optimality reported Unknown.

Hash collisions are resolved by exact comparison. Rationals are canonical (positive denominator, reduced, normalized zero); finite-field values are reduced modulo the declared prime.

An algorithm that computes a distance exactly still has to hand replay a certificate for each end of the interval. Value iteration from zero yields horizon distances, which are lower bounds certified by tests of the same depth, and over all finite words they can approach the distance without reaching it (claim U01). At discount below one, the optimal dual potentials of the Kantorovich programs at the computed fixed point give a `DualPostfixpoint` that closes the interval (U04); at discount one no lower certificate other than tests is registered, so a Precision-0 request whose tests stop short of the upper certificate returns Unknown with the interval, even on a chain with one transition function where the polynomial-time algorithms find the value. Through a declared horizon, a `LayeredPrefixpoint` gives the matching upper bound (U02). Over observation-based policies the trace defect through a horizon is computed by enumerating deterministic policy trees, which suffice because a randomized policy separates no more; a maximizing tree, with the event on which the two laws differ most, is the `PolicyTreeWitness` (O03, O04). Literal congruence is one more group-by: the code value of each state determines the code values of its successors.

### 1.10 Resources and incrementality

Every budget names its units: pair expansions, signature computations, decision-diagram nodes, SAT conflicts, LP pivots, arithmetic bit operations, wall time or memory. A horizon is part of a claim and a budget is not; exhausting a budget yields Unknown with a record of completed work. Host evaluation fuel is never a horizon.

Certificates are cached by the semantic digests of their dependencies. Interface changes reuse them only through the transport rules of the specification, which distinguish universal upper bounds, witnesses, exact values, optimality claims and measure-dependent results; each reuse is recorded as a derivation step citing its rule, and the step validates the certificate's binding first. No rule moves a certificate to another machine, simulation target, stream or snapshot. A result bound to a view keeps the view's queries when the item machine gains a query, and gives the new query's consumer no coverage. Adding a query resumes refinement from $P_{\mathrm{beh}} \cap \ker o$; adding an action resumes from $P_{\mathrm{beh}}$; removing queries or actions transports upper bounds and downgrades coarseness to sufficiency. Restricting to a forward-closed subset transports upper bounds, keeps a witness only if its pair lies in the subset, and recomputes exact values, optimality and measure-dependent results (claims H03, H04). Recompiling a module changes its artifact digest, not its semantic digest, and invalidates nothing.

### 1.11 Estimates and their conversions

An Estimated result carries, in the envelope's `Estimate` field, its estimand, estimator, sampling protocol, sample size, error bound, failure probability, dependence assumptions and combination rule, and also its population (the units the stream samples), its selection rule (which units become rows, and when), the digest of the policy version under which the rows were produced, and the estimand as a function of that population. Reuse under another policy version needs a declared transport argument, for instance pointwise monotone labels on a fixed population, or a new estimate; a selection rule that depends on the policy blocks transport. A stream whose declared digest does not match its file is InvalidInput before any count is taken. A result derived from an estimate keeps the `Estimate` field and is Conditional on `(EstimatedPremise result-digest delta)`, so only a consumer that accepts Estimated results can use it. The $\omega$PLN adapter converts an estimate to evidence counts only when it is a proportion over independent observations of the same estimand; a U-statistic of order two converts to an interval truth value, or to counts of effective size $\lfloor n/2 \rfloor$ for a Hoeffding-type consumer; an anytime-valid or sequentially selected estimate converts to an interval truth value at its stopping time and never to counts; an estimate from dependent samples carries its dependence assumptions. Converting an Estimated result into an exact channel is a negative gate.

## 2 Cetta integration

All MeTTaDinC-specific paths, operation names, drivers and build targets below are proposed additions unless identified as existing. None of them is runnable on unmodified Cetta.

### 2.1 Pin and inspect the host

The inspected repository is `https://github.com/zariuq/CeTTa`, branch `main`, at commit

`12da03b4b055bfecff2add505e7e1ad4125bb352`,

recorded on 14 August 2026. It was still the head of `main` (215 commits) when rechecked on 3 November 2026 [2]. Start from that pin or record the replacement and recheck the seams below.

Existing baseline commands:

```
git clone https://github.com/zariuq/CeTTa.git
cd CeTTa
git checkout 12da03b4b055bfecff2add505e7e1ad4125bb352
make BUILD=core doctor-gmp
make BUILD=core
./cetta --version
./cetta --list-languages
./cetta --list-profiles
make BUILD=core test-correctness test-profiles
make BUILD=core test-rhocalc
```

These are instructions for implementing agents; this guide's authors inspected the source and did not run the suite. Keep three records distinct: upstream-reported test status, a newly executed host baseline, and MeTTaDinC-specific validation.

Use the no-Python core lane unless an adapter needs Python. The plain `make` selects the Python-enabled build (`BUILD ?= python`). GMP is on by default (`ENABLE_GMP ?= 1`); exact profiles verify it at startup and report Unsupported when it is missing, never falling back to floating point. The MORK and PathMap bridges are needed only for snapshot carriers (milestone M7). Cetta's runtime profiles (`-lang he`, `-profile he-extended`, `rhocalc`, `prime`) and MeTTaDinC's profile coordinates are separate namespaces; selecting a host profile certifies nothing about a MeTTaDinC module.

## 2.2 Source map

| Existing source or symbol | Instruction |
|---|---|
| `src/atom.h`, `src/atom.c` | Atom construction and exact numbers (`GV_RATIONAL`, `atom_rational`, GMP helpers) at the boundary. Keep internal identifiers separate from user-visible atoms. |
| `src/term_universe.h`, `src/term_canon.h` | Canonical term storage. Define MeTTaDinC's own binder-aware canonical serialization; host object identity is not a portable semantic identity. |
| `src/library.h`, `src/library.c` | Library context and import paths; native dispatch goes through `cetta_library_dispatch_native`. Follow its error and lifetime conventions. |
| `native/native_modules.h`, `native/native_modules.c` | `CettaNativeBuiltinModule` holds a name, an import bit and a dispatch callback receiving context, space, arena, head, arguments and arity. The static table registers `gparse`; a `dcalc` entry is a proposed addition with its own import bit. |
| `src/native_handle.h` | Handle allocation, lookup, close and cleanup for retained native resources. Handles are process-local references, never certificates. |
| `src/grounded.h`, `src/grounded.c`, `src/eval.c` | `grounded_op_is_type_pure` is a positive capability list; `is_grounded_op` recognizes the private `__cetta_lib_` prefix; `grounded_dispatch_accepts_data_arg` lists data-argument exceptions. Inspect all three when adding a held-syntax entry; name recognition alone does not establish staging. |

| Existing source or symbol | Instruction |
|---|---|
| `lib/stdlib.metta` | `noeval` has type `(-> Atom Atom)` and preserves syntax through application. Test the holding behavior of the chosen `dc.run` wrapper. |
| `src/compile.c`, `src/compile.h` | `compile_space_to_llvm` emits equation-group LLVM with runtime-evaluation fallback. Use it for validated pure wrappers only. |
| `src/gslt_language_runtime.h`, `tools/generate_gslt_language_v1.py` | Generated language packs with the staged `classify/produce/observe` pipeline; the producer closes before its bag is observed. Study `langdef/zero` and `langdef/subzero` before adding a manifest. |
| `src/gslt_compiled_runtime.h` | Source/plan agreement checks and the compiled worklist for the admitted finite-Horn projection; a differential pair with the horn-reference realization. |
| `scripts/rhocalc_bounded_bisimulation.py`, `scripts/rhocalc_bounded_relation_equivalence.py`, `tests/rhocalc_bisimulation.tsv` | Bounded checks for the rho-calculus profile, run from `test-rhocalc`. The bisimulation script performs weak barbed matching: it compares the barbs reachable from each term and searches multistep matching candidates, charging their cost against its depth. Reuse its normalization and successor generation directly; use its verdicts only through the weak-barb adapter of M5. |
| `src/table_store.c` | Answer tabling keyed by space instance identifier and revision. Precedent for in-process snapshot cache keys (M7). |
| `src/mork_space_bridge_runtime.c`, `src/mm2_lower.c` | MORK bridge and MM2 lowering; the route for snapshot carriers and group-by checks over PathMap (M7). |

Proposed layout:

```
lib/dcalc.metta                  public wrappers and constructors
native/dcalc/dcalc_module.[ch]   admission and dispatch
src/dcalc/registry.[ch]          generated from mettadinc_registry.json
src/dcalc/ir.[ch]                term-level typed IR, identities, digests
src/dcalc/elaborate.[ch]         names, types, profiles, defaults
src/dcalc/table.[ch]             enumerated backend
src/dcalc/refine.[ch]            Hopcroft/Paige-Tarjan, splitting trees, lumping
src/dcalc/metric.[ch]            behavioral tolerances, defects, couplings
src/dcalc/trace.[ch]             trace equivalence, trace-event witnesses
src/dcalc/linear.[ch]            Krylov over Q and F_p, realizations, grades
src/dcalc/symbolic.[ch]          decision-diagram backend (narrow interface)
src/dcalc/obligations.[ch]       trusted obligation generator (CNF)
src/dcalc/sat.[ch]               solver calls and proof-log handling
src/dcalc/nondet.[ch]            bisimulation, simulation, formulas
src/dcalc/compose.[ch]           arrow kinds and registered composition laws
src/dcalc/hive.[ch]              kernel and transfer hives, closure, holonomy
src/dcalc/graded.[ch]            kernels, brackets, closure, contracts
src/dcalc/transport.[ch]         certificate cache and transport rules
src/dcalc/result.[ch]            result envelope and schemas
replay/dcalc_replay.[ch]         independent replay checker (separate target)
tests/dcalc/package/             the validation package, unmodified
tests/dcalc/mutations/           adversarial candidates replay must reject
```

Keep the replay checker and the obligation generator in their own build targets with no dependency on the backends, so that they can be reviewed and later verified on their own. Keep the validation package byte-identical to its published version; changes to it are coordinated changes (Section 1.3).

## 2.3 Admission

Implement source admission before any domain algorithm. It consumes held syntax, resolves a complete `DC.Module`, and returns an immutable admitted module or a structured failure; `dc.run` is a wrapper around it. A proposed private entry is `__cetta_lib_dcalc_admit`. Establish raw-data treatment of its arguments through `grounded_dispatch_accepts_data_arg` or an equivalent typed holding wrapper, without broadening data treatment for other native operations and without adding the service to the type-pure list.

The first admission tests install hostile host equations for names such as `Map`, `Body` and a module-local function, and a held expression containing an I/O or state-mutating operator. Loading the module must execute none of them. Phases, in order:

1. parse with source locations, collecting declarations without executing embedded requests;
2. check every declaration and request against the registry: heads, fields, argument kinds, required arguments, defaults; assign module-local identifiers; reject duplicates and unknown references, and never resolve a name against the surrounding space;
3. check the profile coordinates against the capability manifest and the effect and type signatures of imported primitives;
4. check binders, expression types and dependency edges: linear case patterns, one lexical argument per `Body`, an acyclic map-call graph, no derivation cycles;
5. resolve carrier elements at their element types (symbols in symbol-typed carriers, constants otherwise) and reject duplicates under typed equality; compute separation of each value tolerance;
6. for enumerated carriers, materialize maps and kernels, rejecting missing or overlapping cases (even agreeing ones), out-of-carrier values, invalid rationals, unnormalized rows and floating-point literals in exact positions; for symbolic carriers, discharge totality, exclusivity and closure through the obligation generator and the proof pipeline, or by a cited construction rule;
7. freeze the IR and its digest, and execute requests against that snapshot.

Never apply host nondeterministic rewriting to a deterministic map. Two host alternatives are neither a law nor a legal map; a nondeterministic map is admitted only through `Choice` with a declared bag interpretation.

## 2.4 Ownership and persistence

Cetta atoms are arena-allocated with recorded arena identities and reset epochs. Never keep a borrowed pointer from a request's scratch arena in a retained module, cache entry or certificate; copy semantic values into storage whose lifetime covers every consumer, including transitive children. Native resources (decision-diagram managers, solver instances) have one owner and a destructor, and admission defines what happens when a handle closes or a session ends. Three identities stay separate: the semantic module digest, the compiled artifact digest and the runtime handle. Shared admitted modules are immutable; scratch state is thread-local or

follows the host's concurrency protocol. Cancellation releases temporary resources and yields Unknown with the work completed.

### 2.5 Dependencies and entry criteria

Pinned now, in `DEPENDENCIES.json`:

- the CeTTa commit above;
- the proof pipeline: CaDiCaL `rel-3.0.1`, `drat-trim` at `2e3b2dc`, and `cake_lpr` at `033aad5` [22, 23];
- the oracle's Python version; the oracle needs only the standard library.

To be pinned by the milestone that first needs them, before that milestone starts:

- a decision-diagram package (CUDD or Sylvan, or a small in-tree implementation) behind a narrow interface (M3); its results are candidates and replay never depends on it;
- exact linear programming for couplings (M4): transportation problems by an exact network simplex over rationals; replay checks the couplings directly and needs no solver;
- the finite OmegaPLN reference interpreter (M4, differential oracle), the MeTTaProN shared kernel IR (M4, kernel import), CettaPacks (any specialized backend), the MORK and PathMap snapshot interfaces beyond the bridges in the pinned commit (M7), the $\omega$PLN adapter target (M7) and the Hyperseed proof-plan registry (M7).

A pin is an entry in `DEPENDENCIES.json` with repository, commit or release, the interface version used, the license, and a compatibility test that runs in CI. A milestone whose dependency is not pinned may proceed only on the parts that do not touch it, and reports the rest as Unsupported.

### 2.6 Reference path, specialized paths and replay

Build a transparent reference engine before specialized execution: it enumerates the definitions directly—pair graphs, signature refinement, dense lifts—and serves small fixtures and differential tests. Specialized backends must produce semantically comparable results. Replay is not a second run of the optimized routine; it checks equations, couplings, matchings, proof logs, tests, formula graphs and splitting trees against the frozen model. Statuses never collapse: a malformed row is InvalidInput, a valid row exposing a conflict is Refuted, completed exhaustive search supports Checked within its scope, resource exhaustion is Unknown, and an empirical audit is Estimated even when its arithmetic is exact.

### 2.7 Supercompilation, generated packs and the LLVM emitter

Once CettaPacks is pinned, specialized kernels follow its guarded supercompilation design—a multi-geometry IR, a valid/patchable/replan/invalid guard lifecycle, and approximate packs with explicit error witnesses [3]. Initial targets are refinement and splitting-tree traversal, signature construction, sparse block sums, Krylov iteration and finite-field elimination. Generated language packs may orchestrate requests where the finite-Horn projection applies, using the existing generator with a new manifest:

```
python3 tools/generate_gslt_language_v1.py \
  --manifest langdef/dcalc/langdef.metta --source-root langdef \
  --symbol cetta_dcalc_language_v1 \
  --header src/generated/dcalc_language_v1.generated.h \
  --source src/generated/dcalc_language_v1.generated.c \
```

```
  --header-include generated/dcalc_language_v1.generated.h
```

This becomes meaningful only after the manifest and semantic sources exist, and the finite-Horn plan format must not be assumed to call native kernels; that boundary needs its own typed, effect-bounded primitives and reference rules. The LLVM emitter (`./cetta -compile`) suits pure wrapper equations. Measure admission, compilation, execution and replay separately, and compare paths only at equal semantics and arithmetic.

### 2.8 Host acceptance

After changes, run the affected MeTTaDinC targets and the host correctness and profile lanes for the build. Before review, the repository's README recommends `make test-correctness-all test-profiles`; follow its sanitizer, bridge and readiness lanes when those boundaries are reached, and never mark an unavailable external gate as passed. Each milestone delivers a pinned build record, held-syntax admission, exact-number capability checks, the typed IR, its manifest claims, independent replay, structured incomplete and error results, and a source map of every host change.

## 3 Milestones, fixtures and acceptance evidence

Milestones describe capability and evidence, not calendar time. The first useful release is M0–M3: strict certificate boundaries, the full frontend, the graded deterministic core and the symbolic backend. Each work package ends with the manifest claims it covers, reproducible commands, structured results, negative cases, a replay run and a statement of the remaining unsupported surface. A claim is covered when the implementation returns its required status and values with replayable evidence; claims outside the running release must return Unsupported.

### 3.1 M0: strict certificate boundaries

The first milestone is a small implementation whose certificate boundaries are strict. It answers few requests, and it keeps its guarantees under small adversarial changes of its inputs: a premise about the wrong machine, a stream whose content changed, a verdict that rests on a solver answer nobody replayed, a code tolerance that hides a literal failure. The fixtures of the package and of the application bundles already exercise more forms than a first implementation can make reliable, so M0 makes the boundaries strict before any breadth is added. Every later milestone keeps them, and keeps the suite below passing.

M0 delivers one mechanism for each boundary, or fixes its interface where a later milestone builds it.

1. Every certificate is bound to its exact interface. The result envelope carries the binding of the specification's trust model: module digest and subject names, source and target model identities, the checked interface with its value tolerances, and the scope with initial states, reachable-state scope, horizon, discount and policy class. One validation routine runs before any consuming operation uses a premise: `Then` and transfer composition, safety transfer along a simulation, lifting to a keyed store, rebuilds, stream and snapshot reads, and gates. The registry's binding rules run at admission, so a mismatch is InvalidInput naming the field in every release, also for a request that is otherwise Unsupported, and wrong-store lifting and wrong-target safety transfer are rejected before any proof transfer begins.
2. One assurance policy assigns every status, on the grounds of the registry's `assurance` entry.

Conditional results carry their `TrustedBackend` assumptions and their `agreed_by` evidence to every consumer, derivations take the union of their premises' assumptions, the declarations `(Accepts Checked)` and `(Accepts Checked (Conditional (TrustedBackend cvc5)))` are enforced, and a gate counts a Conditional result that its consumer does not accept as inadequate coverage. Results supported by a solver, results replayed independently and empirical estimates stay distinguishable all the way to the consumer.

3. Obligations are built from the admitted source. M0 fixes the obligation generator's interface: its only input is the admitted IR, and every obligation it emits records the digest of that source, so no separately written model of a module stands between the source and its proof obligations; the independent mathematical models of the examples stay as differential checks. It also builds the mutation harness, which edits admitted sources to restore known bugs and requires each affected verdict to change. The generator itself is built in M3, with the symbolic backend, and the access-policy mutation becomes a gate there.

4. Preserving a goal's meaning is an explicit obligation. M0 fixes the result types and the rule: informational refinement and informational equivalence, the one-way and two-way adequacy checks between goal observers, are results under those names, and preservation of the meaning is a separate obligation with declared, protected decoders, $d_{\text{new}}(g_{\text{new}}(x)) = d_{\text{old}}(g_{\text{old}}(x))$ for every valid record $x$; a goal change counts as a renaming only when both hold. The adequacy checks are the `CheckNow` requests of M0. The interpretation request, `CheckInterpretation` of the Omega memory and goals guide, is built with the deterministic core in M2, once its registry patch is adopted.

5. Empirical claims name their populations and selection rules. The envelope's `Estimate` field carries the population, the selection rule, the policy version and the estimand, and a result derived from an estimate is Conditional on it; M0 fixes these records, and the estimators come in M4. A policy change triggers a transport check, which passes only with a declared argument, or a new estimation task; a selection rule that depends on the policy blocks transport.

6. The witness forms and induction preconditions are fixed before the compiler's interfaces harden: the literal-congruence obligation of `InduceExact` with its `CongruenceWitness`; the `PolicyTreeWitness` format with its admissibility check, and Unsupported answers for policy-class trace requests until its replay exists; and the distance certificates of Section 1.7, with Unknown for a Precision-0 measurement whose certificates do not meet.

On the computational side M0 covers the core requests these boundaries touch, on enumerated deterministic modules: `MeasureNow`, `CheckNow`, `CheckUpdates`, `InduceExact`, `CheckAbstractionMap` and `Then`, with the manifest claims E05, E08, E09, H08, H10, H12 and C01–C03. Every other request returns Unsupported or, when a binding rule fails, InvalidInput.

The negative acceptance suite is M0's acceptance. Each entry names the document whose bundle holds the fixture and the status an implementation must return. An entry whose request needs a later milestone is answered Unsupported until that milestone, except that binding failures are InvalidInput from M0 on; from its milestone on it must return the status given.

1. An index of the live-memory guide [20] declared with the item statistic of a machine that has fewer queries than the store's item machine, as in `(Keyed EngineAsItems (Of Items) (Item EngineStat))` where `EngineStat` is a statistic of `EngineTime`, is InvalidInput naming the mismatch. With `(View EngineTime)`, as in the declaration `EngineIndex` of the same module, the statistic is a valid lift, bound to the query `suggest` only.

2. The safety request of the same guide `(CheckSafety NormFirst (Never (Shows`

`unlawful True)) (Via StutterWeighted) (Init live-start))`, whose simulation relates `RealWeighted` to `Weighted`, is InvalidInput; with target `Weighted` it is Refuted.

3. A receipt stream of the WMTM worked example [21], changed while its declared digest stays the same, is InvalidInput, returned before any count or interval, and yields no Estimated value; the example's oracle sets every stale label of `RepliesGate` to zero, and the live-memory guide's oracle does the same to `LiveSuggestions`.
4. The access-policy module of the worked examples [18], edited to restore the auditor bug, gives a Refuted verdict computed from the edited source, with the two-step breach `(assign bob auditor)`, `(assign bob approver)`.
5. The goal `anti-nag` of the Omega memory and goals guide [19], the negation of `nag`, passes both informational adequacy checks, `CheckNow AntiAsNag` and `CheckNow NagForAntiNag`, while its interpretation obligation, `CheckInterpretation NagFollowUps AntiNagFollowUps`, is Refuted at the record `(open 0 ben normal)`, so the change is not a renaming.
6. A bound on the duplicate rate among proposed keys in the WMTM example, reused under a policy that withholds first proposals and without a transport argument, is not accepted; the gate requires a new estimate or a declared argument on a fixed population. The example's oracle computes the counterexample on its own receipts: a policy that withheld every key that is not a duplicate and none that is would propose a subset of the keys of the log-only phase, all of them duplicates. The same oracle checks the argument that carries the bound over to the example's snooze release, under which every key is first proposed at the same cycle as before.
7. Results of the live-memory guide and the WMTM example whose obligations went to cvc5 are Conditional on `(TrustedBackend cvc5 1.4.2 ...)`, with Z3 5.3.3 in `agreed_by`, and so is every result derived from them; a consumer that declares `(Accepts Checked)` counts them as inadequate coverage.
8. In this package, `MeasureNow` and `CheckUpdates` on the literal-congruence fixture are Checked with defect 0, and `InduceExact` is Refuted with a `CongruenceWitness` (C01–C03).
9. In this package, the adaptive-policy fixture has defect 1/2 over words, where the bound 3/4 is Checked, and defect 1 over observation-based policies, where the bound 3/4 is Refuted with a `PolicyTreeWitness` (O01–O04). The first release answers Unsupported, and a release that does not replay policy trees answers Unsupported or Unknown.
10. In this package, the unattained supremum is Unknown at `(Precision 0)`, 7/8 at horizon 3, Refuted against the bound 99/100 by a test of depth 7, and 1/3 at discount 1/2 with a dual post-fixpoint, or Unknown in a release without that certificate kind (U01–U04).

The application bundles carry further negative fixtures from their own counterexamples, listed in their manifests, and the milestone that adds the requests they use adds them to its acceptance.

### 3.2 M1: registry, syntax, staging and result plumbing

*Deliverables.* The frontend of M0 extended to every declaration head, request and library of the registry, generated from or driven by it; parser with source locations; module-local name resolution and imports; profile coordinates and the capability manifest; immutable snapshots and digests; lazy derivations; the result envelope with claims, intervals and the measurement and check status rules; registry-directed `Output` on derivations and result variables; loading of every fixture module of the package.

*Acceptance.* Every fixture module is admitted, or answered Unsupported where its profile or libraries lie outside the release, and serializes identically across runs. Admission agrees with `check_fixtures.py` on the fixtures and on the checker's mutation set: undeclared names (such as a projection from a derivation that does not exist), unknown arguments, wrong reference kinds, missing required arguments, out-of-range rationals, unbound variables, unknown result fields and unimported libraries are InvalidInput. Duplicate names and keyed fields, nonlinear patterns, map-call and derivation cycles, and an `Enum` element that is neither a symbol of a symbol-typed carrier nor a constant of the element type are rejected. Host equations and side effects must not run inside held declarations, and a forged `Checked` atom must gain no privilege.

*Evidence.* Diagnostics, an IR dump and digest for one complete module, the registry digest, and the Unsupported answers for the 78 claims outside the first release.

### 3.3 M2: the graded deterministic core on tables

*Deliverables.* Seeds, value tolerances and separation; the measurement and check requests for current, update, behavioral and trace defects, with both certificates; `Defects`; `BehavioralTolerance` with discount and horizon; `Refine` and `MinimizeForQueries` by Hopcroft or Paige–Tarjan with splitting trees and weakness ledgers; exact and approximate induction; the frontier request `WeakestWithin` on small carriers; `CheckAbstractionMap` with its `Kind`, `Then` and `SamePartition`; the `GradedKernels` library; the clock-rate screen; the reference engine; the replay checker for every certificate kind used; and, once the registry patch of the Omega memory and goals guide is adopted, its `CheckInterpretation` request for the meaning of goal changes.

*Claims.* Eight states E01–E09; receipt packets P01–P11; the deterministic hypothesis modules H01–H12; graded closure G01–G04.

*Negative gates.* Inducing the eight-state first-bit encoding exactly (E05). Reading horizon $h$ as $\Phi^h(1)$, which accepts the constant encoding at horizon 0 and misreports the staged pair (H05–H07). A literal decoder claimed under a trivial value tolerance (H01, H02). A refutation transported to a forward-closed subset (H03, H04). A zero-defect homomorphism reported as a quotient (H12). A check with a bound treated as a measurement, or the reverse (P05, P06). An update leaving its carrier; overlapping cases with equal outputs. A candidate partition that is stable but not coarsest, which replay must reject as a coarseness claim and accept as a sufficiency claim.

*Evidence.* Machine-readable witnesses, splitting trees, ledgers, induced tables and certificates; differential comparison of reference and specialized paths on the fixtures and on a bounded family of random small machines; replay rejecting the mutation set.

### 3.4 M3: symbolic carriers and incrementality

*Deliverables.* Bit-vector and product carriers with term maps; the admission obligations; the obligation generator with its translation-validation suite; the pinned proof pipeline; the cone-of-influence pass; reachability scopes with inductive invariants and reachability words; decision-diagram signature refinement; splitting trees with concrete representatives; `CheckBlockPartition`; the linear backend over $\mathbb{F}_2$ for linear maps; certificate caching and the transport rules.

*Claims.* Forty bits F01–F06. Each `MinimizeForQueries` returns eight blocks—$(x_0, x_1, \pi)$ for the parity machine and $(x_0, x_1, z)$ for the zero machine, where four blocks are single words and four have $2^{38} - 1$ words—with obligations discharged symbolically (or by matrix identities for the parity machine, whose observable space has dimension 3 and unobservable space dimension 37), a seven-test splitting tree with words of length at most two, and eight concrete representatives.

The resource record shows that no request enumerated the carrier.

*Negative gates.* The adversarial zero-tail candidate accepted for the full carrier (F05). A forward-closed over-approximation used to discharge membership without reachability words. A proof log for a different formula than the regenerated one. A CNF from a generator version other than the one recorded. A source mutation that restores a known bug and leaves the verdict unchanged, as in the access-policy example of the worked examples. A solver verdict without a replayed proof reported as Checked. A sampled clock-rate screen with a fixed seed and $10^6$ uniform samples reports zero clock rate for every tail bit of the zero machine (the chance of any nonzero estimate is below $3 \times 10^{-4}$); the four-class encoding it suggests must be refuted with behavioral defect one. The cone-of-influence pass keeps all forty bits of both machines. Incrementality: adding the query $x_1$ resumes refinement from the old partition and matches refinement from scratch; removing `inject` transports upper bounds and downgrades coarseness to sufficiency.

### 3.5 M4: stochastic dynamics, linear realizations and estimates

*Deliverables.* Rational kernels with exact normalization; lumping in $O(m \log n)$; total-variation and Kantorovich defects with couplings; stochastic behavioral tolerances, exact for one transition function and as certified intervals otherwise; modal-test witnesses in shared formula graphs; `CheckTrace` and `MeasureTrace` with trace-event witnesses over words and policy-tree witnesses over observation-based policies, by enumeration of deterministic policy trees through the horizon; layered prefixpoints for horizon bounds and dual post-fixpoints for discounts below one; `TracePartition`; `MinimalLinear` returning a `LinearRealization`; `CheckLinearArrow` with grades and the linear composition law; `PositiveRealization` reporting an interval; finite decision models and policy optimization; Estimated results with estimand, population, selection rule, policy version, estimator, protocol, sample size, $(\varepsilon, \delta)$, dependence assumptions and composition rule, including U-statistics and anytime-valid monitors; kernel import once the MeTTaProN IR is pinned.

*Claims.* The seven-state chain S01–S07, which is mandatory: seven bisimulation classes, six trace classes, the behavioral check of `MergeXY` refuted with the modal test $\varphi(x) = 1/2$, $\varphi(y) = 0$, the trace check passed, 81/200 at discount 9/10, signed minimum five, positive dimension $[5, 5]$. Linear composition L01–L03. The evolutionary archive V01–V04. The regime laboratory R01–R06. The policy classes O01–O04 and the unattained supremum U01–U04 of `certificate_limits.metta`.

*Negative gates.* A trace-event witness offered for the behavioral refutation of `MergeXY`, or the trace check refuted because the behavioral one is (S03, S05). A positive candidate reported as a minimum without a lower bound (S07). Residuals of signed arrows added without the norm factor (L03). The induced trace bound applied without current adequacy. Negative masses, wrong row sums, float probabilities and duplicate outcome keys are InvalidInput. Equal expected rewards with different reward laws are not equal. A same-seed trajectory comparison must not reject the valid three-block quotient. Unbounded trace equivalence is never reported Unsupported in this milestone, and the six-class trace partition is never offered as a Markov quotient. A sampled successor is never a witness. A word witness offered for a policy-class refutation (O03). A Precision-0 measurement reported Checked from tests alone, or an undiscounted value certified by a post-fixpoint (U01). A positive arrow from an unnormalized realization composed with the additive shortcut instead of its norm.

### 3.6 M5: nondeterminism and simulation

*Deliverables.* Nondeterministic maps through `Choice`; bag interpretations; read annotations; bisimulation, simulation preorders and may-trace checks under budgets; Hausdorff-lifted graded distances with two-sided matchings; distinguishing formulas; simulation arrows, safety transfer with initial coverage for the three registered property forms (`Shows`, `EnterFrom`, `ReachWithin`) and the counterexample-guided refinement loop; the weak-barb adapter for Cetta's rho-calculus tests.

*Claims.* Simulation scope H13–H14; the access protocol Y01–Y04; shared and fresh reads D01–D04.

*The rho-calculus adapter.* Cetta's bounded bisimulation script performs weak barbed matching with a cost-charged depth, which is a different relation from the immediate-step bisimulation of the nondeterministic profile, and its depth is not a MeTTaDinC horizon. Reuse its normalization and successor generation as the profile's front end. Define a `WeakBarbed` interface whose observations are the barbs reachable by internal steps and whose transitions are internal-step closures around one visible step, and compare the script's verdicts with that interface only after a test suite establishes how the script's depth corresponds to the adapter's horizon. Until then, the script's verdicts are recorded and not used as a differential oracle.

*Negative gates.* A host result bag used as a law without a scheduler is InvalidInput. Liveness or existential properties claimed through a simulation are Unsupported. A relation that misses one matching move is Refuted with that move as witness. Safety reported Checked through a simulation whose domain misses an initial state (H14). A safety request along a simulation whose abstract machine is not the request's target, which is InvalidInput from M0 on. The script's depth used as a horizon.

### 3.7 M6: composition kernel, hives, merges, graded kernels and contracts

*Deliverables.* Arrow kinds and grades with registered composition laws; transfers as polynomial morphisms with the composite formula; value-pullback certificates as a separate type; kernel hives with measured and unmeasured correspondences, coherence, associators, holonomy, consensus cost, forest completion and small least-coherent-refinement programs; transfer hives with gauges, holonomies and `KernelOf` with and without saturation; merges and translation actions; affine contracts and their composition.

*Claims.* The skill hive and tick library K01–K14; the parity triangle T01–T04; the independent merge P12–P13; affine contracts A01–A03.

*Negative gates.* Composing transfers field-wise in the same direction; reading a checked transfer as a tolerance kernel; saturating only the base member of a cycle, or gauges declared at every member but not intertwined by the transfers (in both cases the block kernel is not transitive); filling an unmeasured correspondence with zero; a merge-dependent claim with unmeasured receipt identities reported anything but Unknown; thresholding treated as equivalence closure; a sensitivity certificate used as an adequacy certificate; affine residuals added without amplification.

### 3.8 M7: Hyperseed, observer discovery and integration

*Deliverables.* Contexts, p-bits with order-named operations, availability sums, schemas, methods, claims, pattern witnesses and graph records; observer families, invariance derivations, `SeekSeparator`, observer sources and `DecideEquivalence`; `ClockRates` with recorded hypotheses; the OmegaPLN differential oracle, the MeTTaProN kernel import, the $\omega$PLN adapter with its conversion rules, and snapshot carriers over MORK or PathMap with digest, revision

and key binding—each only after its dependency is pinned.

*Claims.* Availability W01–W02; patterns Q01–Q02; arithmetic probes X01–X04; neural reports N01–N06; the method M01; the snapshot fixture Z01; the clock-rate screen F07.

*Negative gates.* An unavailable report silently converted to neither; a self-authored claim bypassing checking; a certificate reused after its context changed; an observer evaluated on an unbound global operator; invariance asserted without a derivation; clock-rate exactness claimed without a full-support product law or outside the coordinate library; an Estimated result converted into an exact $\omega$PLN channel, or a U-statistic converted into $n$ independent counts.

### 3.9 Golden results and adversarial mutations

The manifest is the golden specification: 123 claims, each with its fixture form, required status, oracle values and first-release expectation; a claim that needs a replay capability not every release has names it under `until`, with the statuses to return meanwhile. Where a claim names a witness, it is the canonical witness of the registry's `witness_order`; an implementation may return any witness that replays, and conformance compares witnesses by replay, not by equality.

| ID | Module | Request or derivation | Status | R1 | Statement |
|---|---|---|---|---|---|
| E01 | EightState | Defects | Checked | yes | current defect 0, update defect 1, behavioral defect 1 |
| E02 | EightState | CheckUpdates | Refuted | yes | update compatibility fails at (000, 010) under swap |
| E03 | EightState | CheckBehavioral | Refuted | yes | word witness: pair (000,010), continuation swap, answers 0 and 1; bound n-\|X/K_O\| = 6 |
| E04 | EightState | CheckBehavioral | Checked | yes | horizon 0 checks the initial observation only |
| E05 | EightState | InduceExact | Refuted | yes | exact induction fails on the update obligation |
| E06 | EightState | Repair | Checked | yes | four blocks {ab}; ledger 1/2 to 1/4 under the uniform law |
| E07 | EightState | SamePartition | Checked | yes | the repair equals the encoding (a, b) |
| E08 | EightState | AB | Checked | yes | exact quotient on pairs (a,b); the bit c disappears |
| E09 | EightState | Composed | Checked | yes | composite quotient (a,b,c) -> 2a+b |
| F01 | FortyBits | CheckNow | Checked | yes | keeping x0 answers the query now |
| F02 | FortyBits | CheckBehavioral | Refuted | yes | two words differing only in x1, continuation swap |
| F03 | FortyBits | ZeroQuotient | Checked | yes | eight blocks (x0, x1, tail = 0); symbolic certificate; no enumeration |
| F04 | FortyBits | ParityQuotient | Checked | yes | eight blocks (x0, x1, parity); observable dimension 3, unobservable 37 |
| F05 | FortyBits | WholeCarrier | Refuted | yes | the four zero-tail blocks fail coverage of the full carrier; witness word 4 |
| F06 | FortyBits | OnReach | Checked | yes | the same blocks partition Reach(0), which has four words with witnessing words |
| F07 | FortyBitsScreens | ZeroRates | Checked | – | each tail coordinate has mean clock rate 2^-37 under the uniform law |
| P01 | HyperseedMemory | CheckNow | Checked | yes | the posterior encoding answers the posterior query |
| P02 | HyperseedMemory | CheckBehavioral | Refuted | yes | witness (UOnly,VOnly), add-u, posterior 3/4 versus 9/10 |
| P03 | HyperseedMemory | Defects | Checked | yes | defects 0, 3/20, 3/20 |
| P04 | HyperseedMemory | MeasureBehavioral | Checked | yes | discounted behavioral defect 27/200 |
| P05 | HyperseedMemory | CheckBehavioral | Checked | yes | the bound 3/20 holds |
| P06 | HyperseedMemory | CheckBehavioral | Refuted | yes | the bound 1/10 fails; the measurement itself succeeds |

| ID | Module | Request or derivation | Status | R1 | Statement |
|---|---|---|---|---|---|
| P07 | HyperseedMemory | Repair | Checked | yes | four singletons; ledger 3/8 to 1/4 |
| P08 | HyperseedMemory | TwoClass | Checked | yes | {E},{U,V,B} with weakness 5/8 at bound 3/20; optimality Checked by exhaustive search |
| P09 | HyperseedMemory | TwoClassMachine | Checked | yes | representative Both: radius 3/20, arrow defect 3/20 |
| P10 | HyperseedMemory | CheckUpdates | Refuted | yes | merge witness (U,U) and (U,V) merge to U and B |
| P11 | HyperseedMemory | MergeRepair | Checked | yes | with the merge as the only operation, refinement returns the four receipt sets |
| P12 | PacketMerges | CheckImplementation | Checked | – | independent-evidence merge errs by at most 3/20 |
| P13 | PacketMerges | CheckImplementation | Refuted | – | the error 3/20 at (UOnly,UOnly) exceeds 1/10 |
| S01 | SevenState | Bisimulation | Checked | – | seven bisimulation classes |
| S02 | SevenState | TraceClasses | Checked | – | six trace classes; x and y merge |
| S03 | SevenState | Behavioral | Refuted | – | behavioral defect 1/2 with modal witness phi(x)=1/2, phi(y)=0 |
| S04 | SevenState | MeasureBehavioral | Checked | – | d(x,y) = gamma^2/2 = 81/200 |
| S05 | SevenState | Trace | Checked | – | trace laws of x and y agree |
| S06 | SevenState | Linear | Checked | – | minimal signed linear dimension 5 |
| S07 | SevenState | Positive | Checked | – | positive dimension in [5,5]: lower bound from the signed minimum |
| H01 | ObserverEquality | MeasureNow | Checked | yes | defect 0 with Equality Observer; no literal decoder exists |
| H02 | ObserverEquality | InduceExact | Checked | yes | exact up to the observer: decoders land in Y/~0 |
| H03 | ObserverEquality | CheckNow | Refuted | yes | a constant code on red and blue fails |
| H04 | ObserverEquality | CheckNow | Checked | yes | on the forward-closed subset {a} the same code passes |
| H05 | Staged | CheckBehavioral | Checked | yes | horizon 0 |
| H06 | Staged | CheckBehavioral | Checked | yes | horizon 1: still adequate |
| H07 | Staged | CheckBehavioral | Refuted | yes | horizon 2 separates 0 from 1 |
| H08 | Staged | InduceExact | Refuted | yes | the block {0,1} has successors in two blocks |
| H09 | DescentConverse | CheckBehavioral | Checked | yes | behavioral defect 0 |
| H10 | DescentConverse | CheckUpdates | Refuted | yes | update defect 1 |
| C01 | LiteralCongruence | MeasureNow | Checked | yes | current defect 0 under the indiscrete code tolerance |
| C02 | LiteralCongruence | CheckUpdates | Checked | yes | the graded update check passes: defect 0, because the code tolerance identifies every pair of codes |
| C03 | LiteralCongruence | InduceExact | Refuted | yes | literal congruence fails: congruence witness (0,1), same code 0, action hop, successor codes 0 and 1 |
| H11 | ChainBound | CheckBehavioral | Refuted | yes | shortest witness has length n-2 = 4 |
| H12 | Surjectivity | CheckAbstractionMap | Checked | yes | Kind Homomorphism: zero defect, not surjective |
| H13 | SimulationScope | CheckSimulation | Checked | – | the empty relation is a simulation |
| H14 | SimulationScope | CheckSafety | Unknown | – | initial coverage fails; the empty simulation proves nothing |
| O01 | AdaptiveProbe | MeasureTrace | Checked | – | over fixed words of length two the trace defect of {x,x2} is 1/2 |
| O02 | AdaptiveProbe | CheckTrace | Checked | – | the bound 3/4 holds over fixed words |
| O03 | AdaptiveProbe | CheckTrace | Refuted | – | policy-tree witness: a, then b after report 1 and c after report 2; event last report 3; probabilities 1 and 0 |
| O04 | AdaptiveProbe | MeasureTrace | Checked | – | over observation-based policies the trace defect is 1 |
| U01 | UnattainedSupremum | MeasureBehavioral | Unknown | – | d(x,y) = 1 is the supremum of the horizon distances 1-2^-h, attained by no finite test; at Precision 0 the interval does not close |

| ID | Module | Request or derivation | Status | R1 | Statement |
|---|---|---|---|---|---|
| U02 | UnattainedSupremum | MeasureBehavioral | Checked | – | horizon 3: 7/8, from a depth-3 test and a layered prefixpoint |
| U03 | UnattainedSupremum | CheckBehavioral | Refuted | – | a test of depth 7 separates x from y by 127/128 |
| U04 | UnattainedSupremum | MeasureBehavioral | Checked | – | 1/3 from a coupling prefixpoint and a dual post-fixpoint; every test gives (1-4^-h)/3 |
| L01 | LinearComposition | LChecked | Checked | – | grade (Lambda 3, Rho 0) |
| L02 | LinearComposition | KChecked | Checked | – | grade (Lambda 1, Rho 1/5) |
| L03 | LinearComposition | LK | Checked | – | composite grade (3, 3/5), attained; plain addition would give 1/5 |
| G01 | GradedClosure | Increase | Checked | yes | brackets 34/45 and 41/45, increase 7/45 |
| G02 | GradedClosure | CheckTransitive | Refuted | yes | the 9/10 threshold is not transitive |
| G03 | GradedClosure | CheckTransitive | Checked | yes | the closed kernel is metric; raised entry 4/5 |
| G04 | GradedClosure | CheckTransitive | Refuted | yes | the original kernel is not metric |
| A01 | AffineContracts | Composite | Checked | – | composite grade (6, 1/2) |
| A02 | AffineContracts | CheckImplementation | Checked | – | the composite grade holds directly |
| A03 | AffineContracts | CheckImplementation | Refuted | – | unamplified sum fails at input 0 |
| N01 | NeuralAudit | CheckNow | Checked | – | the old report keeps the task |
| N02 | NeuralAudit | CheckNow | Refuted | – | pair (0,0),(0,1) shares report 0 and differs in answer |
| N03 | NeuralAudit | CompareDistinctions | Checked | – | masses 1/2, creation 1/4, erasure 1/4, turnover 1/2 |
| N04 | NeuralAudit | AuditIntervention | Checked | – | accuracy after SetReport 0: old 1/2, new 1 |
| N05 | NeuralAudit | Refine | Checked | – | four singleton classes |
| N06 | NeuralAudit | MinimizeForQueries | Checked | – | two XOR classes |
| V01 | EvolutionArchive | CheckUpdates | Refuted | – | update witness: (G00,G01), mutate, class score = 1, masses 0 and 1/2 |
| V02 | EvolutionArchive | Repaired | Checked | – | three blocks {G00},{G01,G10}, {G11} |
| V03 | EvolutionArchive | ControlledClasses | Checked | – | labelled flips need four classes |
| V04 | EvolutionArchive | Discovery | Checked | – | five proposals in expectation |
| K01 | SkillHive | CheckTransfer | Refuted | – | reward-law defect 1 |
| K02 | SkillHive | CheckValuePullback | Checked | – | optimal value is preserved; not a transfer certificate |
| K03 | SkillHive | GoodCAchecked | Checked | – | defect 0 |
| K04 | SkillHive | RoundTrip | Checked | – | the round trip flips state and method |
| K05 | SkillHive | CheckTransferHive | Checked | – | gauges intertwined by the transfers; holonomy (flip, flip) lies in the gauge |
| K06 | SkillHive | CheckTransferHive | Refuted | – | without a declared gauge the holonomy is a witness |
| K07 | SkillHive | CheckKernelCoherence | Refuted | – | the unsaturated derived kernel hive is the parity triangle |
| K08 | SkillHive | ConsensusCost | Checked | – | closure consensus cost 1/2 |
| K09 | SkillHive | CheckKernelCoherence | Checked | – | saturated by the gauge, the kernel hive is coherent |
| K10 | SkillHiveOriented | CheckTransfer | Refuted | – | an orientation observer breaks the symmetry |
| K11 | TickLibrary | CheckTransfer | Checked | – | transfer defect 0 |
| K12 | TickLibrary | LibraryGap | Checked | – | library gap 1/4 |
| K13 | TickLibrary | CheckTransfer | Refuted | – | expected-reward discrepancy 3/4 |
| K14 | TickLibrary | CheckTransfer | Refuted | – | reward-law total variation 1 |
| T01 | ParityTriangle | CheckKernelCoherence | Refuted | – | holonomy from M1 is the swap |
| T02 | ParityTriangle | Holonomy | Checked | – | the swap kernel |
| T03 | ParityTriangle | ConsensusCost | Checked | – | closure cost 1/2 |
| T04 | ParityTriangle | LeastCoherentRefinement | Checked | – | least coherent refinement costs 1/6 |
| R01 | RegimeRepair | CheckEqual | Checked | – | equal averaged reward laws |
| R02 | RegimeRepair | AverageModel | Checked | – | with prior 3/4 the new averages are 3/4 and 1/4 |

| ID | Module | Request or derivation | Status | R1 | Statement |
|---|---|---|---|---|---|
| R03 | RegimeRepair | InduceExact | Refuted | – | current adequacy fails: the reward-law query separates Plus from Minus |
| R04 | RegimeRepair | OptimizePolicy | Checked | – | coarse optimum 1/2 |
| R05 | RegimeRepair | OptimizePolicy | Checked | – | refined optimum 1 |
| R06 | RegimeRepair | EvaluatePolicy | Checked | – | net value with the sensor 17/20 |
| X01 | ArithmeticProbes | MinimizeForQueries | Checked | – | the Jordan summary merges the two candidates |
| X02 | ArithmeticProbes | Separation | Checked | – | first separator b = 1 with ranks 0 and 1 |
| X03 | ArithmeticProbes | CheckNow | Refuted | – | on the enriched interface the Jordan encoding fails |
| X04 | ArithmeticProbes | Decided | Refuted | – | dim End(K_A) = 6 and dim Hom(K_A,K_B) = 5 |
| Y01 | AccessProtocol | InduceExact | Refuted | – | S0 and S2 share Start but fail leads to Start and Locked |
| Y02 | AccessProtocol | CheckSimulation | Checked | – | the stage map is a simulation |
| Y03 | AccessProtocol | Safe | Checked | – | Admin is entered only from Auth |
| Y04 | AccessProtocol | Lockout | Unknown | – | abstract counterexample Start -> Locked is spurious; refinement splits Start |
| D01 | Reads | Outcomes | Checked | – | outcomes {Agree} |
| D02 | Reads | Outcomes | Checked | – | outcomes {Agree, Differ} |
| D03 | Reads | OutcomeProbability | Checked | – | agreement probability 1 |
| D04 | Reads | OutcomeProbability | Checked | – | agreement probability 1/2 |
| W01 | Availability | CheckNow | Refuted | – | Absent and Neither merge |
| W02 | Availability | CheckNow | Refuted | – | bias also merges Neither and Both |
| Q01 | Patterns | FindCheapestDescription | Checked | – | RepeatAB at cost 4 against 6 |
| Q02 | Patterns | CheckPatternWitness | Checked | – | decodes to the target and beats the baseline |
| Z01 | SnapshotReceipts | CheckNow | Checked | – | holds for any snapshot content, since the code is the query; tests admission of a keyed snapshot carrier and map |
| M01 | HyperseedMethods | Repaired | Checked | – | the method returns an ExactAbstraction of the four-singleton repair |

Table 5: The claims manifest, generated from `manifest.json`. R1 marks the claims the first release must answer; the others must return Unsupported in that release.

Keep source modules and structured results beside it, not transcript fragments. Mutations target semantic mistakes rather than mirroring implementation code: keep a set of intentionally wrong candidate outputs—a stable but non-coarsest partition, a coupling with a wrong marginal, a proof log for another formula, the zero-tail blocks offered for the full carrier, a transfer composed in the wrong direction, a hive with a zero-filled correspondence, a gauge saturated at one member only, a signed composite residual computed by addition, a trace-event witness offered for a behavioral claim, a simulation without initial coverage, a word witness offered for a policy-class claim, a congruence claim decided through the code tolerance, a Conditional result relabelled Checked, a premise bound to another machine—and confirm that replay rejects each one. Test schema round-trips and stale dependencies separately from mathematical obligations.

### 3.10 Handoff and definition of done

Each handoff names the specification date, the registry digest, the Cetta commit, the profile coordinates, owned files, dependencies and the manifest claims covered. A completed package includes:

1. implemented source forms and request signatures, with unsupported forms listed;
2. the manifest claims covered, with their structured results and replay evidence;

3. a build record with toolchain and dependency versions, matching `DEPENDENCIES.json`;
4. correctness, negative, translation-validation and replay results;
5. unresolved assumptions and host limitations;
6. a short account of changed behavior and its evidence.

Proposed targets, absent from unmodified Cetta until implemented:

```
make test-dcalc-boundaries   # M0 negative acceptance suite and binding rules
make test-dcalc-registry     # check_fixtures.py and admission parity (M1)
make test-dcalc-core         # M0-M2 manifest claims
make test-dcalc-symbolic     # M3 claims and the adversarial candidate
make test-dcalc-obligations  # translation validation of the generator
make test-dcalc-proofs       # cadical -> drat-trim -> cake_lpr on all obligations
make test-dcalc-stochastic   # M4
make test-dcalc-nondet       # M5
make test-dcalc-compose      # M6
make test-dcalc-hyperseed    # M7
make test-dcalc-replay       # replay checker against golden and mutation sets
make test-dcalc-oracle       # run the oracle and compare with the manifest
make bench-dcalc
```

Performance work follows semantic acceptance and reports state or description size, action and query counts, nonzero transitions, arithmetic sizes, decision-diagram sizes, proof-log sizes and memory. A speedup obtained by weakening scope, dropping multiplicities, replacing exact laws by floats or skipping replay is a different experiment and is labelled as one.

## References

[1] *MeTTaDinC: An Executable Language for Observer-Relative Abstraction, Evidence, and Composition.* Specification draft, 4 November 2026, with its validation package.

[2] CeTTa contributors. CeTTa source repository, `https://github.com/zariuq/CeTTa`, commit `12da03b4b055bfecff2add505e7e1ad4125bb352`.

[3] B. Goertzel. Design notes on CeTTa-hosted guarded supercompilation (CettaPacks, multi-geometry IR) and approximate CettaPacks. July 2026.

[4] J. Hopcroft. An $n \log n$ algorithm for minimizing states in a finite automaton. In *Theory of Machines and Computations*, Academic Press, 1971, 189–196.

[5] R. Paige and R. E. Tarjan. Three partition refinement algorithms. *SIAM Journal on Computing* 16(6):973–989, 1987.

[6] R. Smetsers, J. Moerman and D. N. Jansen. Minimal separating sequences for all pairs of states. In *LATA 2016*, LNCS 9618, 181–193.

[7] A. Valmari and G. Franceschinis. Simple $O(m \log n)$ time Markov chain lumping. In *TACAS 2010*, LNCS 6015, 38–52.

[8] S. Derisavi, H. Hermanns and W. H. Sanders. Optimal state-space lumping in Markov chains. *Information Processing Letters* 87(6):309–315, 2003.

[9] D. Chen, F. van Breugel and J. Worrell. On the complexity of computing probabilistic bisimilarity. In *FoSSaCS 2012*, LNCS 7213, 437–451.

[10] Q. Tang and F. van Breugel. Computing probabilistic bisimilarity distances via policy iteration. In *CONCUR 2016*, LIPIcs 59, 22:1–22:15.

[11] G. Bacci, G. Bacci, K. G. Larsen and R. Mardare. On-the-fly exact computation of bisimilarity distances. In *TACAS 2013*, LNCS 7795, 1–15.

[12] W.-G. Tzeng. A polynomial-time algorithm for the equivalence of probabilistic automata. *SIAM Journal on Computing* 21(2):216–227, 1992.

[13] S. Kiefer, A. S. Murawski, J. Ouaknine, B. Wachter and J. Worrell. Language equivalence for probabilistic automata. In *CAV 2011*, LNCS 6806, 526–540.

[14] M. R. Henzinger, T. A. Henzinger and P. W. Kopke. Computing simulations on finite and infinite graphs. In *FOCS 1995*, 453–462.

[15] T. Wißmann, S. Milius and L. Schröder. Explaining behavioural inequivalence generically in quasilinear time. In *CONCUR 2021*, LIPIcs 203, 32:1–32:18.

[16] R. Wimmer, M. Herbstritt, H. Hermanns, K. Strampp and B. Becker. Sigref: a symbolic bisimulation tool box. In *ATVA 2006*, LNCS 4218, 477–492.

[17] C. Hensel, S. Junges, J.-P. Katoen, T. Quatmann and M. Volk. The probabilistic model checker Storm. *International Journal on Software Tools for Technology Transfer* 24:589–610, 2022.

[18] *MeTTaDinC in Practice: Four Worked Examples.* Companion to the MeTTaDinC specification draft, 4 November 2026, with its validation bundles.

[19] *Omega Memory and Goals in MeTTaDinC: A Practical Implementation Guide.* Companion to the MeTTaDinC specification draft, 4 November 2026, with its validation bundle.

[20] *Omega Memory and Goals in MeTTaDinC, Part II: Live Memory through Summary Statistics.* Implementation guide, 4 November 2026, with its validation bundle.

[21] *Omega Memory and Goals in MeTTaDinC: WMTM's Open-Item Store, from Journal to Gate.* Worked example, 4 November 2026, with its validation bundle.

[22] N. Wetzler, M. J. H. Heule and W. A. Hunt. DRAT-trim: efficient checking and trimming using expressive clausal proofs. In *SAT 2014*, LNCS 8561, 422–429. Source: `https://github.com/marijnheule/drat-trim`.

[23] Y. K. Tan, M. J. H. Heule and M. O. Myreen. Verified propagation redundancy and compositional UNSAT checking in CakeML. *International Journal on Software Tools for Technology Transfer* 25(2):167–184, 2023. Binaries: `https://github.com/tanyongkiam/cake_lpr`.